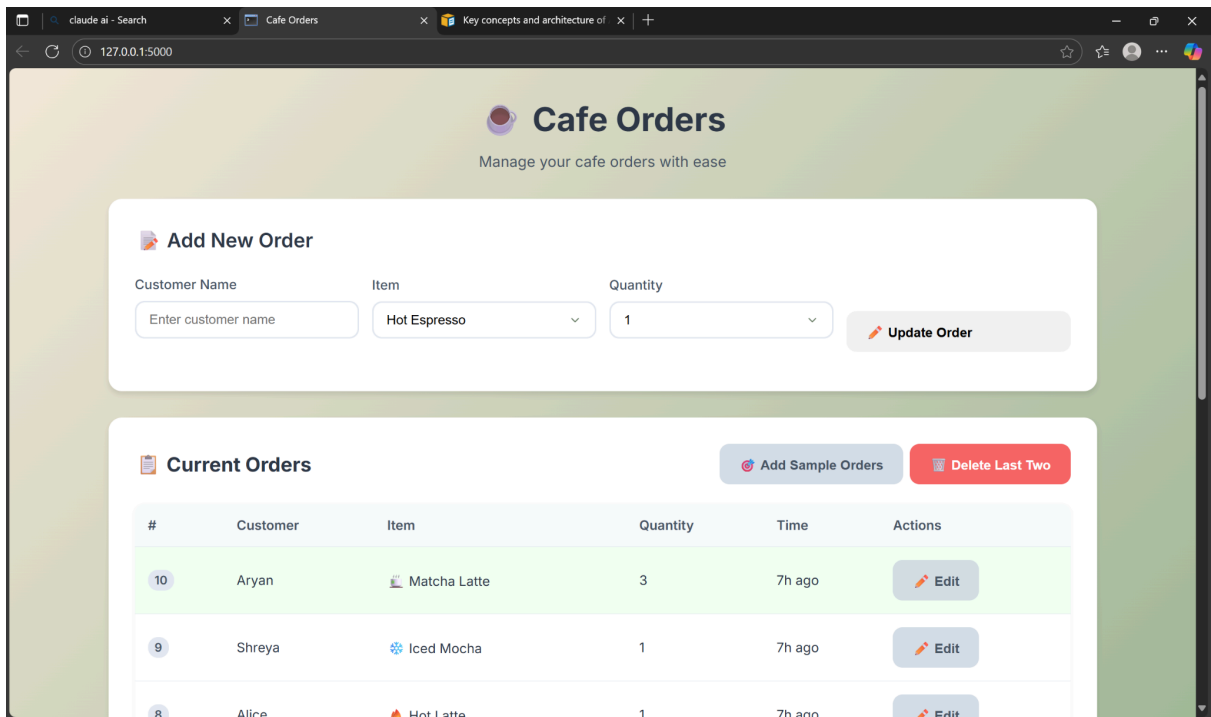


```
PS C:\Users\shrey\OneDrive\Desktop\cafe-app> python -u "C:\Users\shrey\OneDrive\Desktop\cafe-app\app.py"
[+] Database initialized successfully!
[*] Serving Flask app 'app'
[*] Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
[*] Running on all addresses (0.0.0.0)
[*] Running on http://127.0.0.1:5000
[*] Running on http://192.168.10.35:5000
Press CTRL+C to quit
[*] Restarting with stat
[+] Database initialized successfully!
[*] Debugger is active!
[*] Debugger PIN: 104-120-688
```



Browser tabs: claude ai - Search, Cafe Orders, Key concepts and architecture of

Address bar: 127.0.0.1:5000

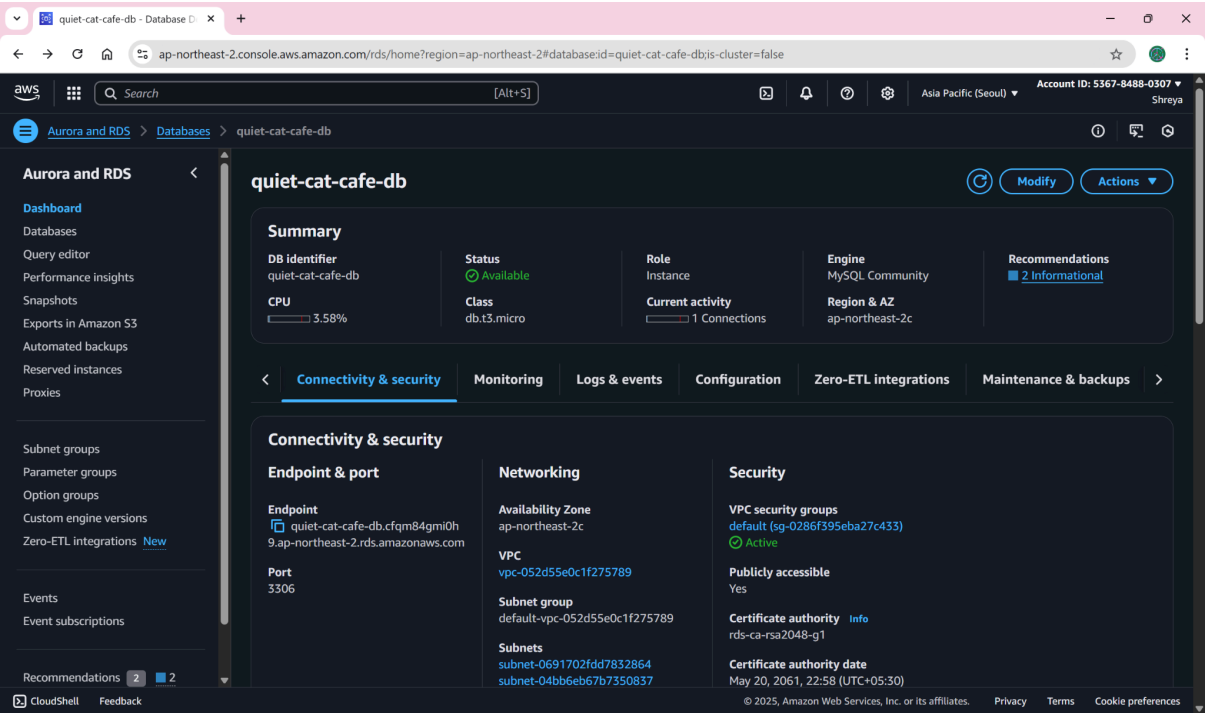
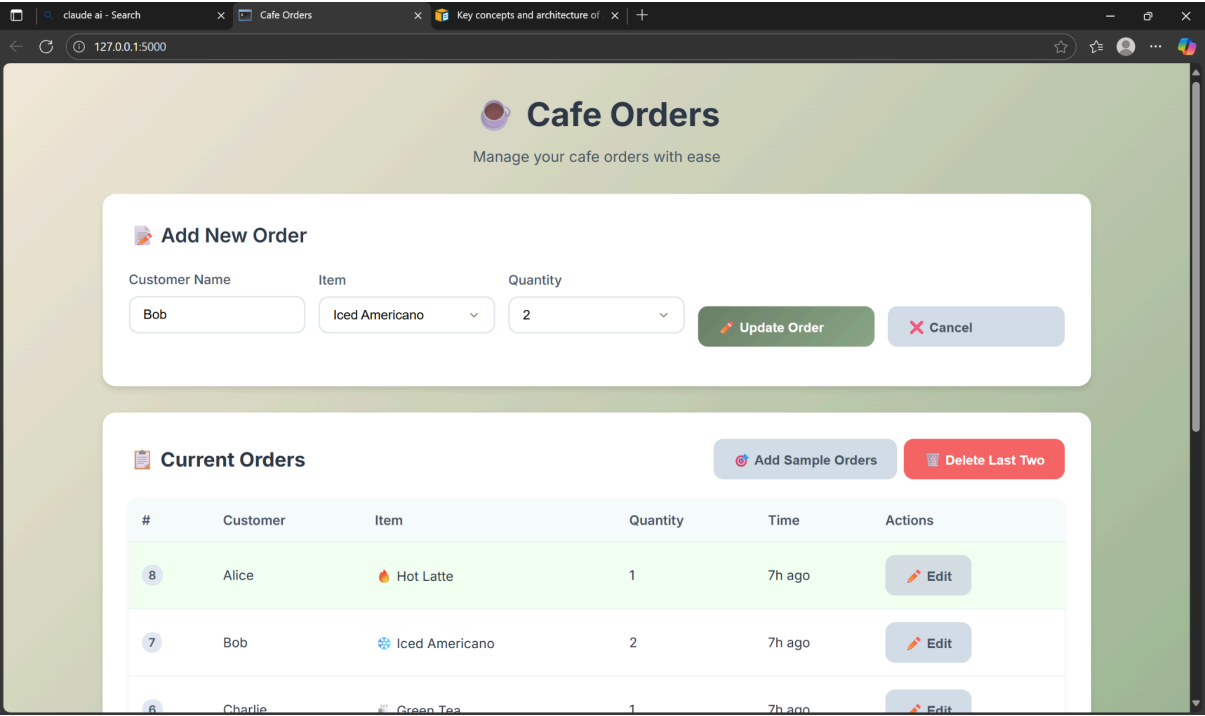
10	Aryan	Matcha Latte	3	7h ago	Edit
9	Shreya	Iced Mocha	1	7h ago	Edit
8	Alice	Hot Latte	1	7h ago	Edit
7	Bob	Iced Americano	2	7h ago	Edit
6	Charlie	Green Tea	1	7h ago	Edit
5	Diana	Hot Cappuccino	1	7h ago	Edit
4	Alice	Hot Latte	2	7h ago	Edit
3	Bob	Iced Americano	2	7h ago	Edit
2	Charlie	Green Tea	1	7h ago	Edit
1	Diana	Hot Cappuccino	1	7h ago	Edit

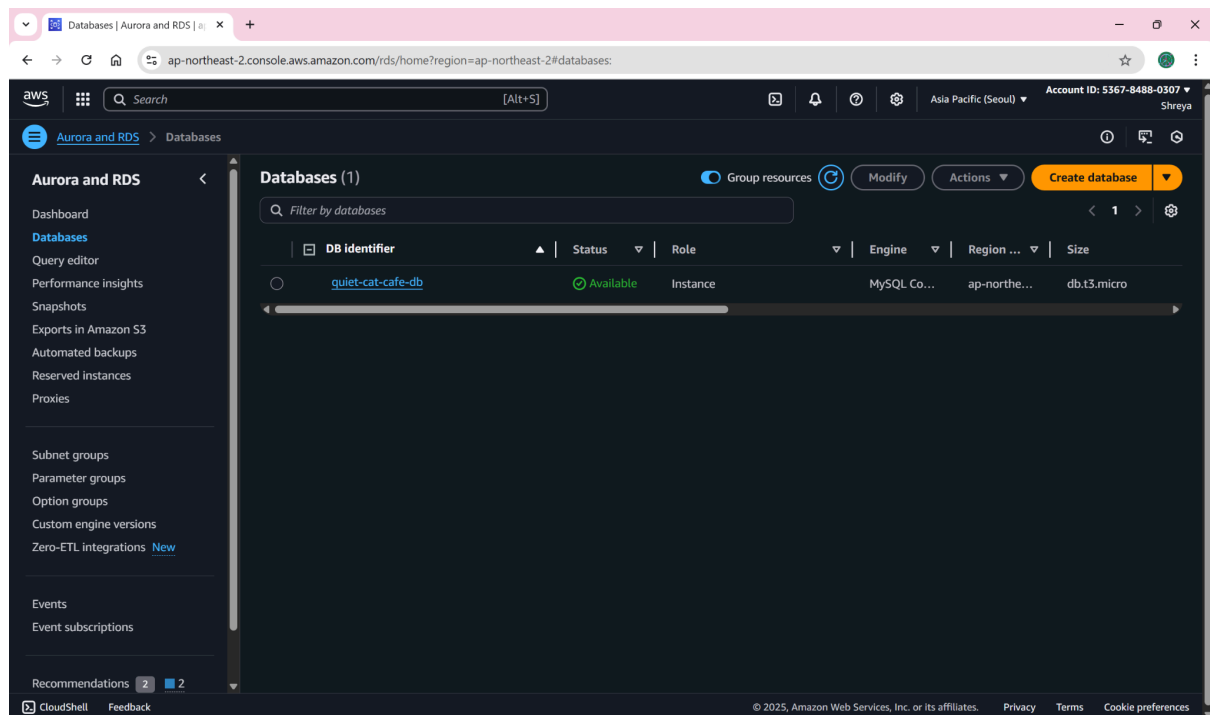
Browser tabs: claude ai - Search, Cafe Orders, Key concepts and architecture of

Address bar: 127.0.0.1:5000

#	Customer	Item	Quantity	Time	Actions
8	Alice	Hot Latte	1	7h ago	Edit
7	Bob	Iced Americano	2	7h ago	Edit
6	Charlie	Green Tea	1	7h ago	Edit
5	Diana	Hot Cappuccino	1	7h ago	Edit
4	Alice	Hot Latte	2	7h ago	Edit
3	Bob	Iced Americano	2	7h ago	Edit
2	Charlie	Green Tea	1	7h ago	Edit
1	Diana	Hot Cappuccino	1	7h ago	Edit

Buttons: Add Sample Orders, Delete Last Two Orders





```
from flask import Flask, request, jsonify, render_template
from datetime import datetime
import os
import mysql.connector
from mysql.connector import Error
import time

app = Flask(__name__)

# MySQL RDS configuration
db_config = {
    'host':
'quiet-cat-cafe-db.cfqm84gmi0h9.ap-northeast-2.rds.amazonaws.com',
    'user': 'admin',
    'password': 'aG+%c8J)QDzrDth',
    'database': 'quiet_cat_cafe',
    'port': 3306,
    'connect_timeout': 10,
    'use_pure': True,
    'autocommit': True,
    'buffered': True
}

def get_db_connection():
```

```

    """Get a database connection with retry logic"""
    retries = 3
    last_error = None

    for attempt in range(retries):
        try:
            conn = mysql.connector.connect(**db_config)
            if conn.is_connected():
                # Configure session
                cursor = conn.cursor()
                cursor.execute('SET SESSION wait_timeout=60')
                cursor.execute('SET SESSION interactive_timeout=60')
                cursor.close()
                return conn
        except Error as e:
            last_error = e
            print(f"Error connecting to MySQL (attempt {attempt + 1}/{retries}): {e}")
            if attempt < retries - 1:
                time.sleep(1)
            else:
                raise last_error

def init_db():
    """Initialize the database and create tables if they don't exist"""
    conn = None
    cursor = None
    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # Drop existing table if it exists
        cursor.execute("DROP TABLE IF EXISTS cafe_orders")

        # Create orders table
        cursor.execute("""
            CREATE TABLE cafe_orders (
                id INT AUTO_INCREMENT PRIMARY KEY,
                customer_name VARCHAR(100) NOT NULL,
                item VARCHAR(100) NOT NULL,
                quantity INT NOT NULL DEFAULT 1,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
    except Error as e:
        print(f"Error initializing database: {e}")

```

```

        ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci
        """
        conn.commit()
        print(" Database initialized successfully!")

    except Error as e:
        print(f" Error initializing database: {e}")
        raise e
    finally:
        if cursor:
            cursor.close()
        if conn and conn.is_connected():
            conn.close()

# Initialize the database on startup
init_db()
deactivate

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/api/orders', methods=['GET'])
def get_orders():
    """Get all orders, sorted by creation time"""
    conn = None
    cursor = None

    try:
        conn = get_db_connection()
        cursor = conn.cursor(dictionary=True)

        # Test connection
        cursor.execute('SELECT 1')
        cursor.fetchone()

        # Get orders
        cursor.execute('''
            SELECT
                id,
                customer_name,
                item,

```

```

        quantity,
        created_at
    FROM cafe_orders
    ORDER BY created_at DESC
'''

orders = cursor.fetchall()
if orders is None:
    raise Exception("Failed to fetch orders from database")

# Format orders for JSON response
formatted_orders = []
for order in orders:
    formatted_order = {
        'id': order['id'],
        'customer_name': order['customer_name'],
        'item': order['item'],
        'quantity': order['quantity'],
        'created_at': order['created_at'].isoformat() if
order['created_at'] else None
    }
    formatted_orders.append(formatted_order)

return jsonify(formatted_orders)

except Exception as e:
    print(f"Error getting orders: {e}")
    return jsonify({
        "error": f"Failed to fetch orders: {str(e)}",
        "details": "Database error. Please try again."
    }), 500

finally:
    if cursor:
        cursor.close()
    if conn and conn.is_connected():
        conn.close()

@app.route('/api/orders', methods=['POST'])
def add_order():
    """Add a new order"""
    conn = None
    cursor = None

```

```

try:
    data = request.get_json()
    if not data:
        return jsonify({"error": "No data provided"}), 400

    # Validate required fields
    required_fields = ['customer_name', 'item', 'quantity']
    for field in required_fields:
        if field not in data:
            return jsonify({"error": f"Missing required field: {field}"}), 400

    # Validate data types and values
    if not isinstance(data['quantity'], int) or data['quantity'] <= 0:
        return jsonify({"error": "Quantity must be a positive integer"}), 400

    if not data['customer_name'].strip():
        return jsonify({"error": "Customer name cannot be empty"}), 400

    if not data['item'].strip():
        return jsonify({"error": "Item cannot be empty"}), 400

    conn = get_db_connection()
    cursor = conn.cursor()

    # Add the order
    cursor.execute(
        'INSERT INTO cafe_orders (customer_name, item, quantity) VALUES (%s, %s, %s)',
        (data['customer_name'].strip(), data['item'].strip(), data['quantity'])
    )
    conn.commit()

    return jsonify({
        "message": "Order added successfully",
        "order_id": cursor.lastrowid
    }), 201

```

```

except Exception as e:
    print(f"Error adding order: {e}")
    return jsonify({"error": f"Failed to add order: {str(e)}"}),
500

finally:
    if cursor:
        cursor.close()
    if conn and conn.is_connected():
        conn.close()

@app.route('/api/sample-orders', methods=['POST'])
def add_sample_orders():
    """Add sample orders for testing"""
    conn = None
    cursor = None

    try:
        sample_orders = [
            ("Alice", "Hot Latte", 1),
            ("Bob", "Iced Americano", 2),
            ("Charlie", "Green Tea", 1),
            ("Diana", "Hot Cappuccino", 1)
        ]

        conn = get_db_connection()
        cursor = conn.cursor()

        cursor.executemany(
            'INSERT INTO cafe_orders (customer_name, item, quantity)
VALUES (%s, %s, %s)',
            sample_orders
        )
        conn.commit()

        return jsonify({
            "message": "Sample orders added successfully",
            "count": len(sample_orders)
        }), 201

    except Exception as e:
        print(f"Error adding sample orders: {e}")

```

```

        return jsonify({"error": f"Failed to add sample orders:
{str(e)}"}), 500

    finally:
        if cursor:
            cursor.close()
        if conn and conn.is_connected():
            conn.close()

@app.route('/api/orders/<int:order_id>', methods=['PUT'])
def update_order(order_id):
    """Update an existing order"""
    conn = None
    cursor = None

    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "No data provided"}), 400

        # Validate required fields
        required_fields = ['customer_name', 'item', 'quantity']
        for field in required_fields:
            if field not in data:
                return jsonify({"error": f"Missing required field:
{field}"}), 400

        # Validate data types and values
        if not isinstance(data['quantity'], int) or data['quantity'] <=
0:
            return jsonify({"error": "Quantity must be a positive
integer"}), 400

        if not data['customer_name'].strip():
            return jsonify({"error": "Customer name cannot be empty"}),
400

        if not data['item'].strip():
            return jsonify({"error": "Item cannot be empty"}), 400

        conn = get_db_connection()
        cursor = conn.cursor()

```

```

        # Check if order exists
        cursor.execute('SELECT id FROM cafe_orders WHERE id = %s',
(order_id,))
        if not cursor.fetchone():
            return jsonify({"error": "Order not found"}), 404

        # Update the order
        cursor.execute(
            'UPDATE cafe_orders SET customer_name = %s, item = %s,
quantity = %s WHERE id = %s',
            (data['customer_name'].strip(), data['item'].strip(),
data['quantity'], order_id)
        )
        conn.commit()

        return jsonify({
            "message": "Order updated successfully",
            "order_id": order_id
        }), 200

    except Exception as e:
        print(f"Error updating order: {e}")
        return jsonify({"error": f"Failed to update order: {str(e)}"}),
500

    finally:
        if cursor:
            cursor.close()
        if conn and conn.is_connected():
            conn.close()

@app.route('/api/delete-last-two', methods=['POST'])
def delete_last_two():
    """Delete the last two orders"""
    conn = None
    cursor = None

    try:
        conn = get_db_connection()
        cursor = conn.cursor()

        # First get the IDs to delete
        cursor.execute("""

```

```

        SELECT id FROM cafe_orders
        ORDER BY created_at DESC
        LIMIT 2
    """)
    ids_to_delete = [row[0] for row in cursor.fetchall()]

    if not ids_to_delete:
        return jsonify({"message": "No orders to delete"}), 200

    # Then delete them
    cursor.execute("""
        DELETE FROM cafe_orders
        WHERE id IN (%s)
    """, % ', '.join(['%s' % id for id in ids_to_delete]), ids_to_delete)

    conn.commit()

    return jsonify({
        "message": f"Successfully deleted {len(ids_to_delete)}
orders",
        "deleted_ids": ids_to_delete
    }), 200

except Exception as e:
    print(f"Error deleting orders: {e}")
    return jsonify({"error": f"Failed to delete orders:
{str(e)}"}), 500

finally:
    if cursor:
        cursor.close()
    if conn and conn.is_connected():
        conn.close()

if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True)

```